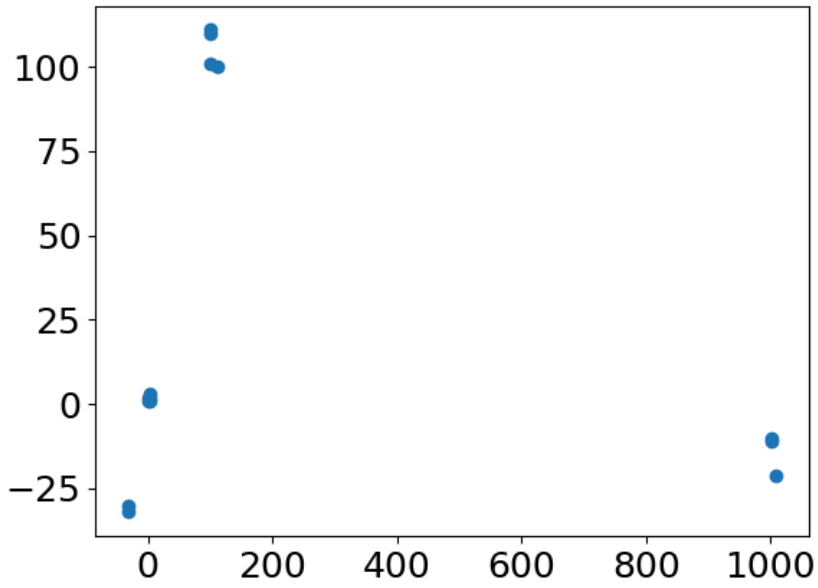
Teme Greedy

1. **Greedy clustering (max 3 stud).** Avem coordonate pentru n puncte și știm că există un număr de k clustere (grupuri de puncte). Să se determine coordonatele celor k centre ale clusterelor și distanța minimă delta dintre un centru și cel mai îndepărtat punct din acel cluster.
   Exemplu de situație: avem n case și vrem să construim k școli/spitale/magazine. Ne interesează să știm unde ar trebui construite cele k clădiri pentru a fi aproape de cât mai multe case.
   Date de intrare: n, k, perechi de coordonate pentru n puncte
   Date de ieșire: k perechi de puncte, distanța delta
   Exemplu:
   n = 14, k = 4
   Punctele: (100, 110), (1001, -10), (1, 1), (1001, -11), (-11, -12), (101, 111), (2, 1), (1, 2), (1010, -21), (2, 3), (111, 100), (101, 101), (2, 2), (-13, -10)

   Pentru a vedea mai clar ce ne cere problema, punctele așezate sub formă de cluster sunt următoarele:
   - (-33, -30), (-31, -32)
   - (1, 1), (1, 2), (2, 1), (2, 2), (2, 3)
   - (100, 110), (101, 101), (101, 111), (111, 100)
   - (1001, -11), (1001, -10), (1010, -21)



Câteva soluții posibile:

- Cele k puncte: (100, 110), (1010, -21), (-11, -12), (2, 3), delta: 14.866
- Dacă punctele sunt inițial în altă ordine, putem obține (1001, -10), (-13, -10), (101, 111), (2, 3)

Observație: chiar dacă avem punctele ordonate, nu știm câte puncte sunt în fiecare cluster! Tocmai asta vrem să aflăm. **Ideea de rezolvare:**

- Se alege un punct inițial la întâmplare
- Cât timp avem mai puțin de k puncte alese, următorul punct este ales ca cel mai îndepărtat punct față de punctele deja alese

Variante:

- Determinist: opțional, se pot sorta punctele la început
- Randomizat: se rulează de câteva ori și inițial se amestecă punctele

Resurse:

- **Pentru algoritm:** CMSC 451: Greedy: The k-Center Problem - UMD
- Clustering and greedy algorithms — Part 2 - Cornell
- Facility location problem - Wikipedia

2. **Patience Sorting (max 3 stud).** Se bazează pe ideile din jocurile de cărți de tip Solitaire. Alte denumiri/variante: Patience, Klondike solitaire, Spider solitaire.

   Avem un șir de n numere pe care vrem să îl sortăm. Considerăm că fiecare număr este o carte de joc și așezăm cărțile în grămezi (stive) crescătoare, cu obiectivul de a obține cât mai puține grămezi:

   - Prima dată creăm o grămadă cu prima carte
   - Pentru fiecare carte rămasă, căutăm să așezăm cartea peste o carte de valoare mai mică; dacă nu există grămezi peste care să putem așeza cartea, creăm o nouă grămadă
   - Grămezile ar trebui ordonate după ultima carte așezată: pentru a găsi rapid grămada în care trebuie să adăugăm următorul element, folosim căutare binară.

   La final vom avea m grămezi, iar fiecare grămadă va avea cărțile sortate descrescător. Puteți folosi std::stack (din <stack>) din C++ pentru implementarea fiecărei grămezi și puteți reține toate stivele într-un std::vector<std::stack<int>>. Pentru partea de căutare binară, nu ar trebui std::vector, ci ar trebui folosită o structură de date care să țină stivele ordonate: de exemplu, std::set sau std::priority_queue (vezi mai jos).

Date de intrare: un număr natural n urmat de o secvență de n numere întregi (nu sunt unice, se pot repeta).
Date de ieșire: cele n numere sortate.

Exemplu:
Avem n = 20 și numerele 20 13 6 10 9 8 15 1 2 16 12 5 11 3 18 19 14 17 7 4.
Se vor efectua următorii pași:

- se face prima grămadă: 20 13 6
- 10 > 6, deci se face o nouă grămadă: 10 9 8
    - 9 și 8 se adaugă la grămada cu 10 deoarece sunt mai mari decât 6
- 15 > 8, 15 > 6, deci se face o nouă grămadă: 15
- În acest punct avem următoarele grămezi (ar trebui sortate):
    - 20 13 6
    - 10 9 8
    - 15
- 1 se va adăuga la prima grămadă, deoarece este cea mai mică grămadă la care putem adăuga
- 2 se va adăuga la a doua grămadă, deoarece este cea mai mică grămadă la care putem adăuga
- La final vom avea următoarele grămezi (am colorat diferit când se schimbă grămada):
    - 20 13 6 1
    - 10 9 8 2
    - 15 12 5 3
    - 16 11 7 4
    - 18 14
    - 19 17

Opțional: pasul următor este să interclasăm grămezile. Este o generalizare la merge sort: acolo aveam 2 șiruri sortate, aici avem m șiruri sortate.

Pentru a interclasa eficient cele m șiruri, ar trebui să folosim o coadă cu priorități (std::priority_queue din <queue>) în care introducem stivele, iar funcția comparator va întoarce vârful stivei. Cât timp avem elemente în coadă, scoatem stiva cea mai prioritară, eliminăm vârful stivei, apoi punem la loc noua stivă în coadă (dacă stiva nu este acum vidă).

Resurse:

- Longest increasing subsequence - Princeton
- Patience sorting - Wikipedia

3. **Generare matrice (max 2 stud).** Se citesc din fișier două numere naturale n și k. Generați o matrice cu n linii și n coloane care să îndeplinească simultan următoarele condiții:
   - conține toate numerele naturale de la 1 la n^2, fiecare număr luat o singură dată
   - pe fiecare linie elementele sunt așezate crescător de la stânga la dreapta
   - suma elementelor de pe o coloană k să fie minimă

   Observație: problema nu are exemple de date de intrare pentru că este simplă.

4. **Optimal offline cache (max 3 stud).** Prin caching (sau memoizare) salvăm niște valori sau rezultate obținute printr-un proces costisitor de care avem nevoie ulterior și nu dorim să le obținem și ulterior de la zero.
   De exemplu, texturile folosite frecvent într-un joc vor fi cache-uite în RAM pentru a nu fi citite de pe disc de fiecare dată.

   Totuși, zona folosită pentru caching este mult mai mică decât totalitatea datelor pe care le accesăm, deci nu putem ține totul în cache. Atunci când un item nu se găsește în cache, avem un cache miss. Avem nevoie de un algoritm care să decidă ce items să țină în cache pentru a minimiza numărul de cache misses.

   O variantă mai simplă a problemei este cea de offline cache pentru care știm în avans ce resurse vor fi accesate, în ce ordine și cât de des.
   Un algoritm Greedy pentru această problemă elimină din cache elementul de care va fi nevoie cel mai târziu.

   Date de intrare:
   - un număr n de resurse, numele resurselor
   - un număr k cu numărul de resurse din cache
   - elementele pe care le conține cache-ul inițial
   - un m număr de accesări de resurse
   - lista cu ordinea în care sunt accesate acele resurse

   Date de ieșire: Numărul de cache misses și pentru fiecare dintre cele m accesări, lista de items din cache la fiecare pas.
   Exemplu:
   Avem 3 resurse: a, b și c.
   Cache-ul are dimensiune 2 și conține inițial elementele a și b.
   Avem următoarele 7 cereri de resurse: a, b, c, b, c, a, b.
   - Pas 1: se cere a, avem în cache a și b
   - Pas 2: se cere a, avem în cache a și b

- Pas 3: se cere c, eliminăm a din cache, avem în cache c și b
    - Îl eliminăm pe a deoarece de b și c mai avem nevoie mai devreme
- Pas 4: se cere b, avem în cache c și b
- Pas 5: se cere c, avem în cache c și b
- Pas 6: se cere a, eliminăm c din cache, avem în cache a și b
    - Îl eliminăm pe c deoarece nu mai apare în viitoarele cereri
- Pas 7: se cere b, avem în cache a și b

În total, avem 2 cache misses.

Resurse:

- Greedy Algorithms I - Princeton
- 16-5 Off-line caching - CLRS Solutions

5. **Stable matching (max 3 stud).** Avem niște candidați și niște firme. Fiecare candidat are o listă de preferințe cu firmele la care ar vrea să facă practică. Fiecare firmă are o listă de preferințe cu studenții pe care i-ar dori (nu întrebăm de unde au aceste liste).
Dorim să găsim toate perechile de (student, firmă) astfel încât să nu existe perechi instabile: studentul A să fie la firma X deși voia la firma Y, iar firma Y să îl aibă pe studentul B, deși l-ar fi preferat pe A.

Problema se poate rezolva cu un algoritm Greedy. Cât timp există studenți fără oferte acceptate, au loc mai multe runde.
În cadrul unei runde, fiecare firmă face o ofertă celui mai preferat student, iar fiecare student fie consideră oferta și o poate accepta, fie respinge oferta dacă are deja o ofertă mai bună.

Date de intrare: un număr n de studenți și un număr n de firme. Pentru fiecare student, avem lista de preferințe cu firmele (în ce ordine preferă ofertele), iar pentru fiecare firmă avem lista de preferințe cu studenții.
Problema se poate rezolva și cu un număr mai mare de firme, însă nu este necesar. În realitate rămân posturi neocupate.
Date de ieșire: n perechi student-firmă.
Exemplu: Avem studenții q, r, s, t și firmele A, B, C, D.

- q preferă A, B, C, D
- r preferă A, D, C, B
- s preferă B, A, C, D
- t preferă D, B, C, A
- A preferă t, s, r, q
- B preferă r, t, q, s
- C preferă t, r, s, q
- D preferă s, r, q, t

Au loc următoarele runde:

- A îi face o ofertă lui t, t consideră oferta
- B îi face o ofertă lui r, r consideră oferta
- C îi face o ofertă lui t; t consideră oferta și are acum o ofertă mai bună, deci respinge oferta lui A
- D îi face o ofertă lui s
- A l-a pierdut pe t, deci îi face o ofertă lui s; s respinge oferta lui D
- B și C momentan nu fac nimic, studenții către care au făcut oferte încă nu au primit alte oferte mai bune
- D l-a pierdut pe s, deci îi face o ofertă lui r; r respinge oferta lui B
- A nu l-a pierdut pe s, deci nu face nimic
- B l-a pierdut pe r, deci îi face o ofertă lui t; t respinge oferta lui C
- C îi face o ofertă lui r; lui r nu îi convine, deoarece a fost acceptat de D, deci respinge oferta lui C
- D nu l-a pierdut pe r, deci nu face nimic
- A nu l-a pierdut pe s, deci nu face nimic
- B nu l-a pierdut pe t, deci nu face nimic
- C îi face o ofertă lui s; s respinge oferta lui C (preferă mai mult A)
- D nu l-a pierdut pe r, deci nu face nimic
- A nu l-a pierdut pe s, deci nu face nimic
- B nu l-a pierdut pe t, deci nu face nimic
- C îi face o ofertă lui q; q nu a mai primit altă ofertă, deci acceptă
- D nu l-a pierdut pe r, deci nu face nimic
- Toți studenții au oferte, algoritmul se încheie cu perechile A-s, B-t, C-q, D-r

Resurse:

- Greedy Algorithms - Illinois
- Gale–Shapley algorithm - Wikipedia

6. **Colorarea grafurilor (max 2 stud).** Se dă un graf. Să se coloreze toate nodurile grafului folosind cât mai puține culori astfel încât oricare două noduri vecine să aibă culori diferite.

   Presupunem că avem o ordine fixă a tuturor culorilor. Algoritmul Greedy ia nodurile la rând o singură dată. Pentru fiecare nod, algoritmul alege prima culoare disponibilă care să nu fie luată deja de un vecin. O variantă este să parcurgem nodurile în ordinea numărului de vecini, colorând întâi nodurile cu cei mai mulți vecini; altă variantă este să colorăm întâi nodurile cu cei mai puțini vecini necolorați.

   Date de intrare: liste de adiacență cu vecinii pentru fiecare nod.
   Date de ieșire: graful colorat și numărul de culori.

Exemplu: Graful este dat de următoarele liste de adiacență:

- 1: 2, 3, 4, 5
- 2: 1, 5
- 3: 1, 5
- 4: 1, 6
- 5: 1, 2, 3, 7
- 6: 4, 8, 9
- 7: 5, 10
- 8: 6
- 9: 6, 10, 11
- 10: 7, 9, 13
- 11: 9, 12
- 12: 11, 13
- 13: 10, 12

Câteva posibile colorări (nod-culoare):

- 1-1, 2-2, 3-2, 4-2, 5-3, 6-1, 7-1, 8-2, 9-2, 10-3, 11-1, 12-2, 13-1
- 1-1, 2-3, 3-3, 4-2, 5-2, 6-1, 7-3, 8-2, 9-2, 10-1, 11-1, 12-2, 13-3

Total: 3 culori

Variantă: vecinii oricărui nod să aibă culori distincte. Pentru această variantă, nodurile ar trebui parcurse începând cu nodurile care au cei mai mulți vecini, colorând întâi nodul respectiv (dacă nu este colorat), apoi toți vecinii săi, verificând ca pentru fiecare nod colorat, vecinii deja colorați să aibă culori diferite. Pe același graf de mai sus, s-ar colora astfel:

- Se colorează 1-1 (are 4 vecini), apoi vecinii săi: 2-2, 3-3, 4-4, 5-5
- Se alege nodul 5 (are tot 4 vecini); e deja colorat, se colorează doar vecinii necolorați, adică 7: 7-4
- Se alege nodul 6 (3 vecini): se colorează cu 2 (prima culoare disponibilă), apoi se colorează vecinii: 8-1, 9-3
- Se alege nodul 9 (3 vecini): e deja colorat, se colorează vecinii: 6 e deja colorat; pentru nodul 10, se colorează cu 1 (culorile folosite sunt vecinii colorați ai lui 7 și ai lui 9, deci culorile 4, 5, 3, 2)
- Se alege nodul 10: nodul 10 și vecinii 7 și 9 sunt colorați, se colorează nodul 13 cu 2 (culorile deja folosite sunt nodul 10 și vecinii săi, deci culorile 1, 4, 3)
- Se aleg acum nodurile cu 2 vecini: 2, 3, 4, 7, 11
- Nodul 11 nu e colorat; culorile deja folosite sunt nodul 9 și vecinii săi, deci culorile 3, 1, 2; colorăm 11-4
- Se alege nodul 12 (2 vecini); culorile deja folosite sunt nodurile 11 și 13 și vecinii acestora, deci culorile 2, 4, 1, 3; colorăm 12-5
- Se alege nodul 13 (2 vecini), apoi nodul 8 (1 vecin)
- Final: 1-1, 2-2, 3-3, 4-4, 5-5, 6-2, 7-4, 8-1, 9-3, 10-1, 11-4, 12-5, 13-2, total 5 culori

Resurse:

- Lecture 9 - Washington
- Lecture 24: Graph coloring and the greedy algorithm - Kennesaw
- Greedy coloring - Wikipedia

7. **Greedy Set Cover (max 3 stud - variante).** La o școală pot merge copii din mai multe orașe din apropiere. Pentru n orașe și m județe, să se determine numărul minim de școli de construit (max o școală/județ) astfel încât să fie deservite toate cele n orașe. Pentru fiecare județ, știm mulțimea de orașe care pot fi deservite.

   Algoritmul Greedy este următorul: la fiecare pas se alege județul în care, dacă am construi o școală, s-ar deservi cel mai mare număr de orașe care nu a fost deja deservit de o altă școală din alt județ.
   Variante: numărul maxim de orașe, buget de construcție, distanțele dintre orașele deservite de un județ.

   Date de intrare: pe primul rând un număr n de orașe și un număr m de județe, iar pe următoarele m rânduri numerele orașelor deservite de județul i (rândul i). Dacă vă ajută, puteți avea la început de rând numărul de orașe din județ.
   Date de ieșire: numărul minim de orașe și numele/id-ul acestora.
   Exemplu: 12 orașe și 7 județe. Fiecare județ poate deservi orașele de mai jos:
   - J1: 1, 4, 7
   - J2: 2, 5, 8
   - J3: 3, 6, 9
   - J4: 1, 2, 3, 4, 5, 6
   - J5: 5, 6, 8, 9
   - J6: 10, 11, 12
   - J7: 8, 9, 11

   Soluția Greedy va alege (în această ordine) județele J4, J6, J5, J1 și este optimă. Pe caz general, algoritmul Greedy nu ajunge la soluția optimă, dar se poate demonstra că soluția dată de Greedy este “destul de bună” și că nu putem obține o soluție mai bună cu un algoritm de aceeași complexitate ca algoritmul Greedy.

   Exemplu: 12 orașe și 6 județe.
   Soluția optimă alege județele J1, J2, J3. Greedy alege județele J4, J6, J5, J1.
   - J1: 1, 4, 7, 10
   - J2: 2, 5, 8, 11
   - J3: 3, 6, 9, 12
   - J4: 1, 2, 3, 4, 5, 6
   - J5: 5, 6, 8, 9
   - J6: 10, 11, 12

   Resurse:
   - 2.1 Greedy Set Cover - Wisconsin
   - CMSC 451: Lecture 9 Greedy Approximation: Set Cover - UMD
   - Greedy Algorithm for Set Cover - Dartmouth
   - Greedy algorithms - Berkeley

8. **Partiționarea intervalelor (max 3 stud).** Avem n activități pentru care știm ora de început și ora de sfârșit. Avem posibilitatea să folosim oricât de multe săli, însă dorim minimizarea numărului de săli pentru a scădea costurile administrative. Două activități care se suprapun trebuie planificate în săli diferite.

   Numărul minim de săli este numărul maxim de săli ocupate în orice moment.

   Pentru o implementare eficientă, aveți nevoie de o stivă (de exemplu vector simplu sau std::stack). Intervalele sunt reținute ca evenimente de start și de sfârșit. Inițial, stiva conține n săli libere (de ce?). La evenimente de start eliminăm o sală din stivă, iar la evenimente de sfârșit punem la loc sala în stivă.

   Date de intrare: numărul n de activități și n perechi de intervale pentru orele de început și de sfârșit.
   Date de ieșire: numărul de săli și pentru fiecare sală ce activități sunt planificate.
   Exemplu: Avem 12 activități de planificat:
   - 1: [13:20, 13:40]
   - 2: [7:30, 9:30]
   - 3: [9:10, 12]
   - 4: [9:30, 13]
   - 5: [8:40, 10]
   - 6: [14, 15]
   - 7: [12:10, 15]
   - 8: [7, 8:30]
   - 9: [8, 9]
   - 10: [13:20, 16]
   - 11: [11:10, 13:10]
   - 12: [10:20, 11]

   Avem nevoie de 3 săli:
   - Sala 1: activitățile 8, 5, 12, 11, 1 și 6
   - Sala 2: activitățile 2, 4 și 10
   - Sala 3: activitățile 9, 3 și 7

   Resurse:
   - [CMSC 451: Lecture 7 Greedy Algorithms for Scheduling]
   - [Greedy Algorithms I - Princeton]

9. **Fracții egiptene (max 2 stud).** Să se aplice algoritmul Greedy pentru a transforma o fracție obișnuită într-o sumă de fracții cu numărătorul 1. Se va aplica succesiv formula următoare:

$$\frac{x}{y} = \frac{1}{\lceil \frac{y}{x} \rceil} + \frac{(-y) \bmod x}{y \lceil \frac{y}{x} \rceil}$$

   Date de intrare: o fracție inițială (cu numărător mai mare de 1).

Date de ieșire: sumă de fracții cu numărător 1.
Exemplu:

$$\frac{7}{15} = \frac{1}{3} + \frac{2}{15} = \frac{1}{3} + \frac{1}{8} + \frac{1}{120}.$$

Resurse:

- Egyptian Fractions - UCI
- Greedy algorithm for Egyptian fractions - Wikipedia

10. **Codificare Huffman (max 3 stud - variante).** Se dă un fișier text care ocupă cel puțin 10KB. Dorim să comprimăm acest fișier pentru a ocupa mai puțin spațiu.

    Codificarea Huffman determină inițial frecvența de apariție a fiecărui caracter în textul pe care îl codificăm. Caracterele care apar cel mai des vor fi codificate cu un cod foarte scurt, iar caracterele care apar de puține ori vor primi un cod mai lung. Codificarea are proprietatea că avem întotdeauna un prefix unic pentru fiecare caracter, deci nu avem ambiguitate la decodificare.

    După determinarea frecvențelor se construiește dicționarul, rezultatul fiind un arbore binar. Inițial, fiecare nod reprezintă numărul de apariții al caracterului. La fiecare pas, creăm un nou nod format din 2 noduri nevizitate cu valorile cele mai mici. Nodul nou creat se consideră nevizitat, iar valoarea este suma celor două noduri inițiale (aceste 2 noduri se marchează ca vizitate). Continuăm acest proces până când nu mai avem noduri vizitate.

    Ultimul pas este codificarea efectivă. Fiecare caracter este scris în fișierul de ieșire folosind codul din dicționar.

    Variante (pentru 2 și 3 ar trebui std::unordered_map):

    - Calculăm frecvența la nivel de caracter (cea de mai devreme și din exemplul de mai jos)
    - Calculăm frecvența la nivel de cuvânt (textul trebuie segmentat după spații și semne de punctuație); nu folosiți texte mari cu multe cuvinte unice!
    - Calculăm frecvența la nivel de bigrame/trigrame, adică grupuri de 2 sau 3 caractere; exemplu: trigramele pentru cuvântul teste sunt tes, est, ste

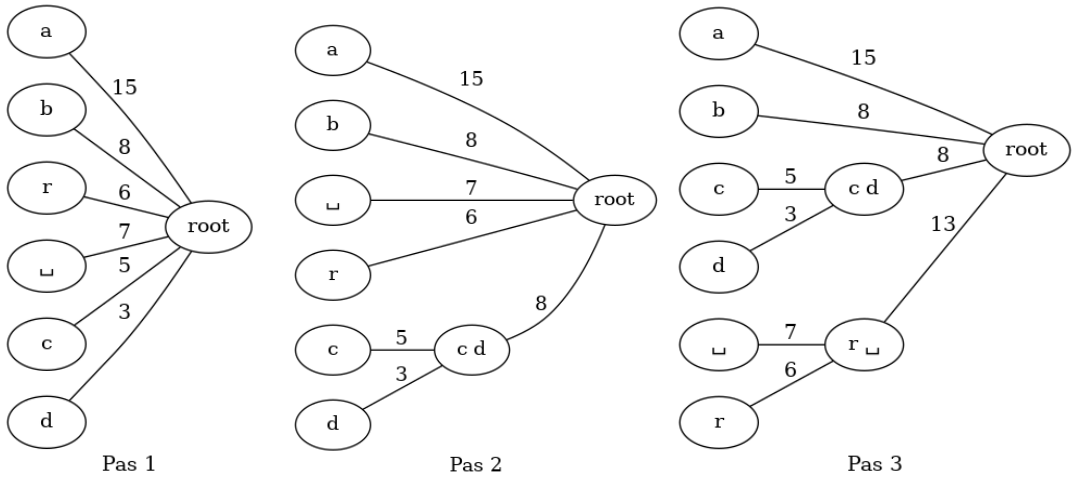
    Date de intrare:

    - Pentru toate variantele: Project Gutenberg
    - Pentru varianta la nivel de caracter, alternativ: Secvențe de ADN/ARN

    Date de ieșire: fișierul comprimat (inclusiv dicționarul) și rata de compresie

Exemplu: avem textul “abracadabra brad barba barca daca ac bac bar”.
Frecvențele caracterelor sunt:

- a 15
- b 8
- r 6
- c 5
- d 3
- ␣ 7 (spațiu)



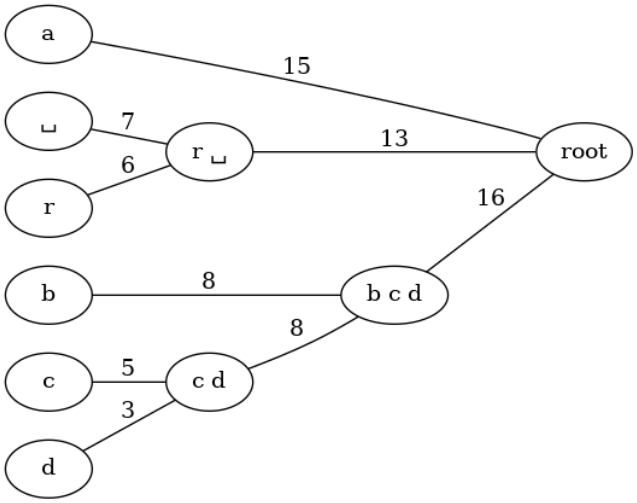
Nodurile cu valorile cele mai mici de la pasul 1 sunt c și d (3 și 5). Creăm un nou nod “c d” care va avea valoarea 8, iar nodurile c și d sunt marcate ca vizitate, deci acum avem frecvențele:

- a 15
- b 8
- r 6
- c d 8
- ␣ 7

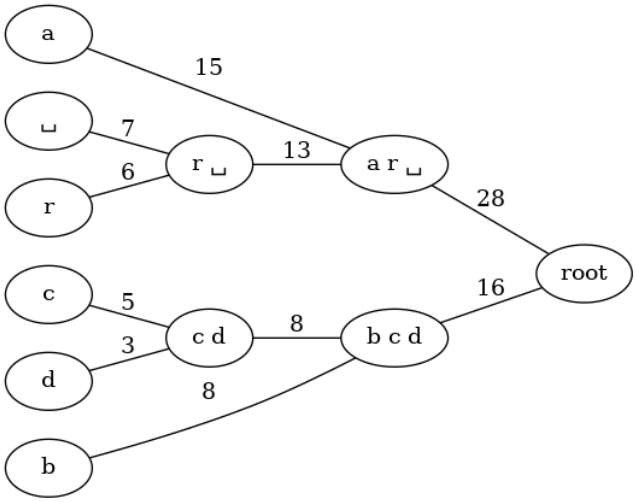
Nodurile cu valorile cele mai mici de la pasul 2 sunt r și ␣ (6 și 7). Creăm un nou nod “r ␣” care va avea valoarea 13, iar nodurile r și ␣ sunt marcate ca vizitate, deci acum avem frecvențele:

- a 15
- b 8
- c d 8
- r ␣ 13

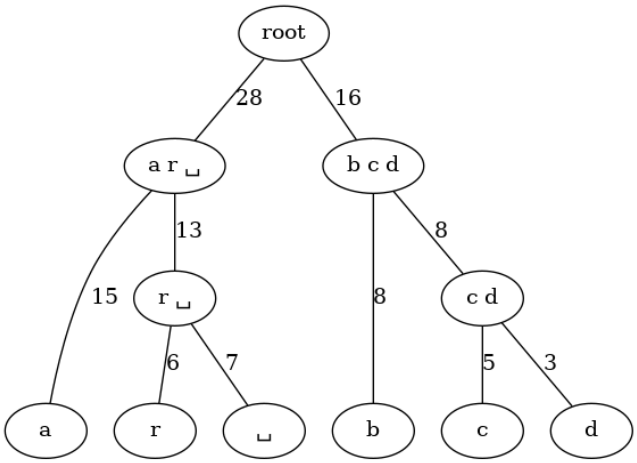
Nodurile de la pasul 3 sunt b și “c d” (8 și 8). Creăm nodul “b c d” de valoare 16 și marcăm nodurile b și “c d” ca vizitate.



Pas 4



Pas 5



Pas 6

În acest punct avem următoarele frecvențe:

- a 15
- b c d 16
- r ␣ 13

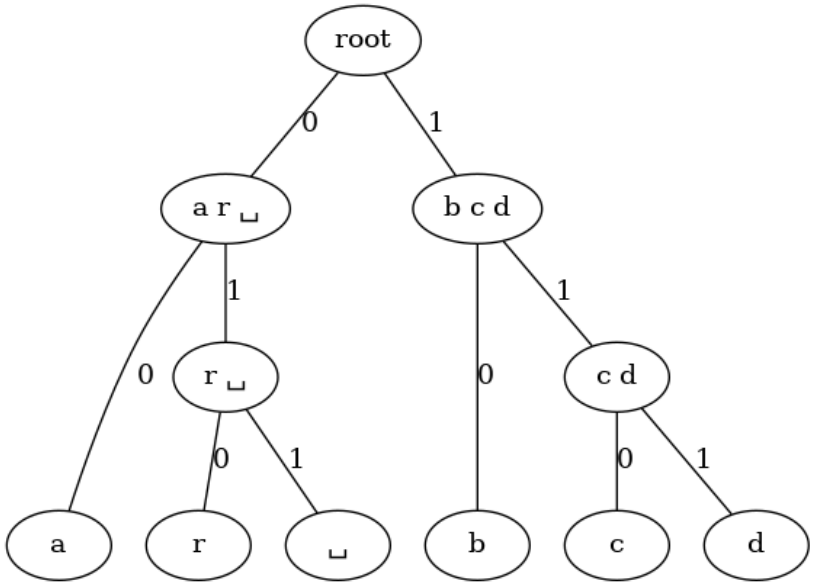
Nodurile cu cele mai mici valori de la pasul 4 sunt a și “r ␣”, iar nodul rezultat are valoare 28.

În acest punct (pasul 5), putem să unim ultimele 2 noduri rămase pentru a crea nodul rădăcină și le marcăm ca fiind vizitate (sau oprim algoritmul dacă este mai convenabil astfel).

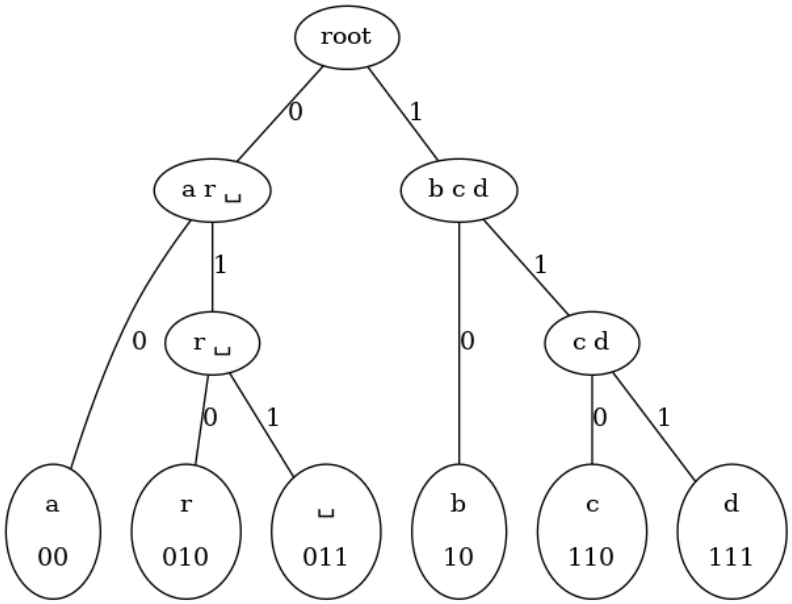
Arborele de la pasul 6 este identic cu cel de la pasul 5, dar este rotit. Acesta este un arbore binar, deci fiecare nod are exact 2 subarbori (fii) sau este un nod frunză.

Notăm muchiile spre fiul stâng cu 0 și muchiile spre fiul drept cu 1. Obținem rezultatul din figura cu pasul 7.

La ultimul pas (pasul 8) obținem codificările pentru fiecare caracter: codificarea este calea de la rădăcină la fiecare frunză în parte, folosind valorile muchiilor de la pasul 7.



Pas 7



Pas 8

De exemplu, pentru a ajunge de la rădăcină la nodul r, din rădăcină avem stânga, dreapta, stânga, deci codul este 010.

Cuvântul “bar” va fi codificat ca 10 00 010 (fără spații).

Textul de la începutul exemplului va fi codificat ca următoarea secvență de 0 și 1: “0010010001100011100100100001110010001110111000010100001110000101100000111110011000011001100111000110011100001 0”.

Acest șir de caractere poate fi comprimat prin unsigned chars, grupând numerele de mai sus câte 8: “00100100”, “01100011”, …, “11001110”, “00010”.

Cu excepția ultimului grup (care este incomplet), putem scrie într-un fișier binar valorile acestor caractere: 36, 99, 146, 28, 142, 225, 67, 133, 135, 204, 51, 56, 206. Vom presupune că nu știm să rezolvăm problema cu ultimul grup, așa că îl vom stoca într-un fișier separat ca atare.

Spațiul ocupat acum este: 13 bytes cu primul fișier și 5 bytes cu al doilea fișier (putem reduce la 2 bytes: 1 byte pentru lungime și 1 byte valoarea efectivă). Textul inițial are 44 de bytes, deci avem o rată de compresie de 40.9% (59.1% spațiu economisit).

Totuși, compresia reală este mai mică, deoarece trebuie să reținem și dicționarul. Dacă stocăm dicționarul ca text, va ocupa 24 bytes. Dacă îl stocăm ca fișier binar (presupunem că avem caractere unsigned char care nu apar mai mult de 2^8-1 ori), va ocupa 12 bytes (6 caractere, 1 byte fiecare caracter + 1 byte nr de apariții). Așadar, rata reală de compresie este 68.2% (31.8% spațiu economisit).

Pentru texte/fișiere de dimensiuni mai mari, ar trebui să obținem o rată de compresie mai bună, însă va trebui să folosim unsigned int în loc de unsigned char la stocarea numărului de apariții în cazul dicționarului.

Resurse:

- Greedy algorithms - Berkeley
- Greedy Algorithms - Illinois
- Huffman coding - Wikipedia